

Culture SPB — Спецификация требований и архитектуры

Версия: 0.3 (draft) Дата: 2026-03-07 Авторы: Clawdributors + С-ЗРО

Содержание

- [Обзор проекта](#1-обзор-проекта)
 - [Глоссарий](#2-глоссарий)
 - [Фаза 1 — Scope](#3-фаза-1--scope)
 - [Модель данных (НСИ)](#4-модель-данных-нси)
 - [Источники данных](#5-источники-данных)
 - [Пайплайн сбора и обработки](#6-пайплайн-сбора-и-обработки)
 - [Автообнаружение источников и bootstrap каталога](#7-автообнаружение-источников-и-bootstrap-каталога)
 - [Telegram-канал (News Feed)](#8-телеграм-канал-news-feed)
 - [Веб-портал](#9-веб-портал)
 - [Админ-интерфейс](#10-админ-интерфейс)
 - [Архитектура системы](#11-архитектура-системы)
 - [Стек технологий](#12-стек-технологий)
 - [Нефункциональные требования](#13-нефункциональные-требования)
 - [Фаза 2+ — Дорожная карта](#14-фаза-2--дорожная-карта)
-

1. Обзор проекта

1.1. Проблема

Информация о выставках изобразительного искусства в Санкт-Петербурге разбросана по десяткам источников: сайты музеев, телеграм-каналы об искусстве, порталы-агрегаторы (Афиша, Собака и т.д.). Нет единого места, где вся эта информация была бы собрана, нормализована и представлена в удобном виде.

1.2. Решение

Culture SPB — платформа-агрегатор, которая:

- Автоматически собирает информацию о выставках из множества источников
- Нормализует её в единый формат с помощью LLM

- Дедуплицирует события
- Публикует в телеграм-канал и на веб-портал
- Поддерживает каталог площадок, художников и выставок (НСИ)

1.3. Целевая аудитория (фаза 1)

Ограниченная группа пользователей — любители искусства Санкт-Петербурга, заинтересованные в оперативном получении консолидированной информации о выставках.

1.4. География и тематика

- **Город:** Санкт-Петербург (архитектура должна допускать расширение на другие города)
- **Тематика:** Выставки изобразительного искусства (живопись, графика, скульптура, фотография, инсталляции, медиа-арт и т.д.)

2. Глоссарий

Термин	Определение
НСИ	Нормативно-справочная информация — каталоги площадок, художников, выставок
Площадка (Venue)	Музей, галерея, выставочное пространство, где проходят выставки
Выставка (Exhibition)	Культурное событие — экспозиция работ в определённой площадке в определённый период
Художник (Artist)	Автор или авторы работ, представленных на выставке
Источник (Source)	Сайт, телеграм-канал или портал, откуда собирается информация
Пайплайн	Автоматический процесс: сбор → фильтрация → нормализация → дедупликация → публикация
Фид (Feed)	Лента нормализованных новостей о выставках

3. Фаза 1 — Score

3.1. Что входит (IN)

#	Компонент	Описание
1	Пайплайн сбора данных	Ежедневный автоматический сбор из всех типов источников
2	LLM-обработка	Определение релевантности, нормализация, тегирование, дедупликация
3	НСИ	Каталоги площадок, художников, выставок
4	Телеграм-канал	Автоматическая публикация нормализованных новостей

#	Компонент	Описание
5	Веб-портал	Read-only портал с каталогом, поиском, календарём, SEO
6	Еженедельный дайджест	«Заканчивается на этой неделе» — воскресный пост + pinned
7	Админ-панель	Управление НСИ, источниками, модерация, статистика
8	Bootstrap-скрипт	Автоматическое обнаружение источников и наполнение первичного каталога

3.2. Что НЕ входит (OUT — фаза 2+)

- Пользовательские аккаунты и авторизация
- Персональные планировщики и календари
- Шеринг календарей между пользователями
- Отзывы и рейтинги
- Карта выставок
- Рекомендательная система
- Мобильные приложения
- Мультигородность

4. Модель данных (НСИ)

4.1. Площадка (Venue)

Поле	Тип	Обязательное	Описание
id	UUID	✓	Уникальный идентификатор
name	string	✓	Название (русское)
slug	string	✓	URL-friendly идентификатор
type	enum	✓	museum / gallery / exhibition_space / cultural_center / other
address	string	✓	Полный адрес
geo_lat	float	Широта (для будущей карты)	
geo_lon	float	Долгота (для будущей карты)	
website	string	URL официального сайта	
description	text	Описание площадки	
working_hours	jsonb	Часы работы (структурированные)	
image_url	string	URL фото площадки	
is_active	bool	✓	Активна ли площадка

Поле	Тип	Обязательное	Описание
created_at	timestamp	✓	
updated_at	timestamp	✓	

4.2. Художник (Artist)

Поле	Тип	Обязательное	Описание
id	UUID	✓	
name	string	✓	Имя (русское)
slug	string	✓	URL-friendly
name_original	string	Имя на языке оригинала (если не русский)	
birth_year	int	Год рождения	
death_year	int	Год смерти (null = жив)	
bio	text	Краткая биография	
image_url	string	URL фото/портрета	
created_at	timestamp	✓	
updated_at	timestamp	✓	

4.3. Выставка (Exhibition)

Поле	Тип	Обязательное	Описание
id	UUID	✓	
title	string	✓	Название выставки
slug	string	✓	URL-friendly
venue_id	FK → Venue	✓	Площадка
date_start	date	Дата начала (null = неизвестна)	
date_end	date	Дата окончания (null = неизвестна / постоянная)	
status	enum	✓	announced / running / ending_soon / ended / cancelled
type	enum	✓	solo / group / permanent / other
description	text	Описание выставки	
description_short	string	Краткое описание (для поста)	
image_url	string	Постер/обложка выставки	
source_urls	string[]	✓	URL-ы источников, откуда получена информация
tags	string[]	Автоматические теги (жанр, стиль, эпоха и т.д.)	

Поле	Тип	Обязательное	Описание
is_published	bool	✓	Опубликована ли (для модерации)
moderation_status	enum	✓	pending / approved / rejected / auto_approved
created_at	timestamp	✓	
updated_at	timestamp	✓	

4.4. Связь Exhibition ↔ Artist (many-to-many)

Поле	Тип	Описание
exhibition_id	FK → Exhibition	
artist_id	FK → Artist	
role	enum	main / participant / curator

4.5. Источник (Source)

Поле	Тип	Обязательное	Описание
id	UUID	✓	
name	string	✓	Человекочитаемое название
type	enum	✓	website_museum / website_aggregator / telegram_channel / rss
url	string	✓	URL сайта или t.me/channel_name
scraper_config	jsonb	Конфигурация парсера (CSS-селекторы, API endpoints и т.д.)	
is_active	bool	✓	Активен ли источник
reliability_score	float	Оценка качества/полезности (0-1)	
last_scraped_at	timestamp	Когда последний раз собирали	
last_success_at	timestamp	Когда последний раз успешно собрали	
items_total	int	Всего событий получено за всё время	
items_relevant	int	Из них релевантных	
created_at	timestamp	✓	

4.6. Сырое событие (RawEvent) — staging-таблица

Поле	Тип	Описание
id	UUID	
source_id	FK → Source	Откуда получено
raw_content	text	Сырой контент (HTML/текст поста)
raw_image_urls	string[]	URL-ы изображений из источника
scraped_at	timestamp	Когда собрано
is_relevant	bool / null	Результат фильтрации (null = ещё не обработано)
relevance_reason	string	Пояснение LLM почему релевантно/нет
normalized_data	jsonb / null	Нормализованные данные (промежуточный результат)
exhibition_id	FK → Exhibition / null	Связь с созданной/обновлённой выставкой
processing_status	enum	new / filtered / normalized / deduplicated / published / rejected
duplicate_of	FK → RawEvent / null	Если дубликат — ссылка на оригинал

4.7. Пост в фиде (FeedPost)

Поле	Тип	Описание
id	UUID	
exhibition_id	FK → Exhibition	К какой выставке относится
type	enum	new_exhibition / update / ending_soon / digest
title	string	Заголовок поста
body_short	text	Краткий текст (для телеграма и карточки)
body_full	text	Полный текст (для веб-страницы)
image_url	string	Изображение поста
telegram_message_id	int / null	ID опубликованного сообщения в Telegram
is_published	bool	
published_at	timestamp / null	
is_pinned	bool	Закреплено ли в канале
created_at	timestamp	

5. Источники данных

5.1. Типы источников

5.1.1. Сайты музеев и галерей (~100–200 площадок)

Примеры: hermitagemuseum.org, ruseum.ru, erarta.com, bofreart.com, и т.д.

Метод сбора: HTTP-скрейпинг + LLM-экстракция.

Для каждого сайта хранится конфигурация:

- URL раздела «Выставки» / «Афиша»
- Метод парсинга: `llm_extract` (страницу целиком отправляем в LLM) или `css_selectors` (структурированный парсинг + LLM для обогащения)
- На этапе bootstrap — начинаем с `llm_extract` для всех, оптимизируем по мере необходимости

Почему LLM-экстракция как дефолт:

- Сайты музеев имеют разную структуру, писать парсер под каждый — дорого
- LLM отлично справляется с извлечением структурированных данных из HTML/текста
- При изменении дизайна сайта LLM-парсер продолжает работать (в отличие от CSS-селекторов)

5.1.2. Телеграм-каналы

Примеры: @spb_afisha, @artpetersburg, каналы конкретных галерей и т.д.

Метод сбора: Telethon (userbot-клиент для чтения публичных каналов).

Процесс:

- Подключаемся к публичным каналам
- Раз в день забираем новые посты за последние 24 часа
- Текст + изображения отправляются в пайплайн обработки

Важно: Используем только чтение публичных каналов. Не вступаем, не взаимодействуем.

User account (не bot API) нужен для чтения каналов без вступления.

5.1.3. Сайты-агрегаторы

Список агрегаторов расширяемый — новые источники добавляются через админку или обнаруживаются автоматически (раздел 7). Ниже — примеры известных агрегаторов для начального bootstrap:

Сайт	Раздел	Метод
kudago.com	Выставки / СПб	REST API (есть публичный)
afisha.ru	Выставки / СПб	HTTP-скрейпинг
sobaka.ru	Культура	HTTP-скрейпинг
timeout.ru	Выставки / СПб	HTTP-скрейпинг
the-village.ru	Город / Культура	HTTP-скрейпинг
kuda-spb.ru	Выставки	HTTP-скрейпинг
allexhibitions.ru	СПб	HTTP-скрейпинг
culture.ru	Афиша / СПб	HTTP-скрейпинг
...		

Полный список формируется на этапе bootstrap (раздел 7) и пополняется через процедуру еженедельного обнаружения. Каждый агрегатор — запись в таблице `source`, которую можно добавить/отключить/удалить через админку.

5.2. Приоритизация источников

Приоритет	Тип	Обоснование
 Высший	Сайты музеев/галерей	Первоисточник, наиболее точные даты и описания
 Средний	Агрегаторы	Широкий охват, но могут содержать ошибки
 Дополнительный	Telegram-каналы	Оперативность, но могут быть неточны; хороши для обнаружения событий

При конфликте данных — приоритет у первоисточника (сайт площадки).

6. Пайплайн сбора и обработки

6.1. Общая схема

Этапы пайплайна (последовательно):

Шаг	Этап	Вход	Выход
1	Сбор (Scrapers)	Источники (сайты, TG-каналы, агрегаторы)	RawEvent (status: new)
2	Фильтрация (LLM Relevance)	RawEvent (new)	RawEvent (filtered , is_relevant = true/false)
3	Нормализация + Тегирование (LLM)	RawEvent (filtered , relevant)	Normalized data + tags
4	Обогащение (Proactive Enrichment)	Normalized data	Обогащённые данные из первоисточника (сайт площадки)
5	Дедупликация (эвристика + LLM)	Normalized data	Exhibition (create / update / merge)
6	Модерация (ручная или auto-approve)	Exhibition (pending)	Exhibition (approved / rejected)
7	Формирование поста (Feed Generation)	Exhibition (approved)	FeedPost
8	Публикация (TG + Web)	FeedPost	Сообщение в канале + обновление сайта

6.2. Этап 1: Сбор (Scraping)

Расписание: Ежедневно, ночью (03:00–05:00 MSK), чтобы все источники обновились за день. **Для каждого активного Source:**

- Скачать контент (HTTP GET / Telethon fetch)
- Сохранить в RawEvent со статусом new
- Сохранить изображения (скачать, положить в object storage)
- Записать last_scraped_at в Source

Обработка ошибок:

- При ошибке HTTP — retry 3 раза с exponential backoff
- При 3+ днях подряд ошибок — пометить Source как проблемный, уведомить в админке
- Rate limiting: пауза 2-5 сек между запросами к одному домену

6.3. Этап 2: Фильтрация релевантности (LLM)

Задача: Определить, относится ли собранный контент к выставкам изобразительного искусства в Санкт-Петербурге. **LLM-промпт (концепт):**

``

Ты — классификатор культурных событий. Определи, относится ли следующий текст к выставке изобразительного искусства в Санкт-Петербурге.

Релевантно: выставки живописи, графики, скульптуры, фотографии, инсталляций, медиа-арта в музеях, галереях и выставочных пространствах Санкт-Петербурга.

НЕ релевантно: концерты, спектакли, лекции (если не привязаны к выставке), события в других городах, продажа произведений искусства (аукционы), мастер-классы без привязки к выставке.

Текст:

{raw_content}

Ответ в JSON:

```
{
  "is_relevant": true/false,
  "confidence": 0.0-1.0,
  "reason": "краткое пояснение"
}
```

` Пороги:

- confidence ≥ 0.8 → автоматически relevant
- confidence < 0.3 → автоматически not relevant
- Между 0.3 и 0.8 → на ручную модерацию (на этапе тестирования — всё на модерацию)

Результат: обновление RawEvent.is_relevant, relevance_reason, статус → filtered

6.4. Этап 3: Нормализация и тегирование (LLM)

Задача: Из сырого контента извлечь структурированные данные в формате Exhibition. **LLM-промпт (концепт):** `

Извлеки информацию о выставке из следующего текста.

Если информация отсутствует – оставь поле null.

Если в тексте несколько выставок – верни массив.

Текст:

{raw_content}

Верни JSON:

```
{
  "exhibitions": [
    {
      "title": "string",
      "venue_name": "string – название площадки",
      "venue_address": "string или null",
      "date_start": "YYYY-MM-DD или null",
      "date_end": "YYYY-MM-DD или null",
      "artists": ["string – имена художников"] или [],
      "description": "string – полное описание",
      "description_short": "string – 1-2 предложения",
      "type": "solo/group/permanent/other",
      "tags": ["string – жанр, стиль, эпоха, техника"],
      "image_description": "string – описание главного изображения, если есть"
    }
  ]
}
```

Правила тегирования:

- Жанр: живопись, графика, скульптура, фотография, инсталляция, медиа-арт, декоративно-прикладное, смешанная техника
 - Стиль (если определим): импрессионизм, авангард, современное искусство, реализм, абстракция и т.д.
 - Эпоха (если определим): античность, средневековье, XVIII век, XIX век, XX век, современность
- Дополнительно:**

- Связывание venue_name с существующей площадкой в каталоге (fuzzy match по названию)

- Если площадка не найдена – создать новую запись со статусом `pending` (требуется подтверждения в админке)
- Связывание художников – аналогично: поиск по имени, создание при отсутствии

Результат: `RawEvent.normalized_data` заполнен, статус → `normalized`

6.5. Этап 4: Проактивное обогащение (Proactive Enrichment)

Задача: Когда информация о выставке получена из вторичного источника (телеграм-канал, агрегатор), система проактивно обращается к первоисточнику – сайту площадки – для получения полных и достоверных данных. **Процесс:**

- После нормализации определяется площадка (`venue_id`)
- Если источник `RawEvent` – не сайт самой площадки (т.е. `Source.type != website_museum`):
 - а. Найти `Source` типа `website_museum` для этой площадки
 - б. Загрузить страницу выставок площадки
 - в. LLM-экстракция: найти на странице информацию о конкретной выставке (по названию / датам)
 - г. Мерж данных: дополнить/уточнить нормализованные данные из первоисточника
 - Если площадка ещё не имеет привязанного `Source` – пометить для ручного добавления

Приоритет данных при мерже:

- Сайт площадки > агрегатор > телеграм-канал
- При конфликте дат – берём с сайта площадки
- Описание – берём самое полное (или мержим)
- Изображение – приоритет у первоисточника (обычно лучшее качество)

Реактивное обогащение (Enrichment при повторных упоминаниях):

Когда новый `RawEvent` при дедупликации сопоставляется с уже существующей `Exhibition`:

- Сравниваем поля: если в новом источнике есть данные, отсутствующие в текущей записи → дополняем
- Если данные конфликтуют → применяем приоритет источников (см. выше)
- Логируем все изменения для аудита
- При существенных изменениях (смена дат, площадки) → создаём `FeedPost` типа `update`

Результат: Обогащённые данные с максимальной полнотой и достоверностью.

6.6. Этап 5: Дедупликация

Двухуровневая стратегия: Уровень 1 – Эвристический (быстрый):

- Совпадение `venue_id` + нечёткое совпадение `title` (`Levenshtein distance ≤ 0.2`) + пересечение дат → вероятный дубликат
- Совпадение `venue_id` + совпадение хотя бы одного художника + пересечение дат → вероятный дубликат

Уровень 2 – LLM (для неочевидных случаев):

Определи, описывают ли эти два текста одну и ту же выставку:

Выставка A: `{title_a}, {venue_a}, {dates_a}, {description_a}`

Выставка B: `{title_b}, {venue_b}, {dates_b}, {description_b}`

Ответ: `{ "is_duplicate": true/false, "confidence": 0.0-1.0, "reason": "..."}`

При обнаружении дубликата:

- Мержим данные: дополняем недостающие поля из нового источника
- Добавляем `source_url` в массив источников выставки
- `RawEvent.duplicate_of` → ссылка на оригинал
- Статус → `deduplicated`

6.7. Этап 6: Модерация

Режим тестирования (начальный):

- Все нормализованные события попадают в очередь модерации
- Модератор в админке: `Approve / Reject / Edit & Approve`
- При `Approve` → `Exhibition.moderation_status = approved`, `is_published = true`

Рабочий режим (после стабилизации):

- Автопубликация для событий с `relevance_confidence ≥ 0.8` от надёжных источников (`Source.reliability_score ≥ 0.7`)
- Остальное — в очередь модерации
- Возможность скрыть/отредактировать любую выставку через админку в любой момент

6.8. Этап 7: Формирование постов (Feed Generation)

Типы постов:

Тип	Триггер	Расписание
new_exhibition	Новая выставка одобрена	После модерации / автоматически
update	Существенное обновление (изменение дат, площадки)	После обнаружения
ending_soon	Выставка заканчивается в течение 7 дней	Воскресенье, утро
digest	Еженедельный дайджест «Заканчивается на этой неделе»	Воскресенье, 10:00 MSK

Формат нормализованного поста (для Telegram):`

 {Название выставки}

 {Площадка}

 {Дата начала} – {Дата окончания}

 {Художник(и)} (если есть)

{Краткое описание – 2-3 предложения}

 {теги через #}

 Подробнее: {canonical_url на веб-портале}

`Формат дайджеста «Заканчивается на этой неделе»:

 Заканчиваются на этой неделе

-  {Название} – до {дата}

 {Площадка}

-  {Название} – до {дата}

 {Площадка}

...

Не пропустите! Полный список: {url}

Дайджест публикуется и закрепляется (pin) в канале. Предыдущий дайджест открепляется.

6.9. Этап 8: Публикация

Telegram:

- Публикация через Telegram Bot API
- Пост: изображение (если есть) + текст + inline-кнопка «Подробнее» (ссылка на веб)
- Дайджест: текст + pin

Веб-портал:

- Регенерация статических страниц после завершения пайплайна (SSG build)

- Обновление поискового индекса

6.10. Автоматическое обновление статусов выставок

Ежедневный cron job:

- announced → running (если date_start <= today)
- running → ending_soon (если date_end - today <= 7 days)
- ending_soon → ended (если date_end < today)

7. Автообнаружение источников и bootstrap каталога

Разовый процесс при запуске проекта + еженедельное обновление.

7.1. Шаг 1: Seed-каталог площадок

Входные данные: LLM-запрос + веб-поиск. **Процесс:**

- Запрос к LLM: «Составь максимально полный список музеев, галерей, выставочных пространств Санкт-Петербурга, где проходят выставки изобразительного искусства. Для каждого укажи: название, адрес, сайт (если знаешь), тип (музей/галерея/выставочное пространство).»
- Серия web search запросов:

- "галереи современного искусства Санкт-Петербург"

- "музеи Санкт-Петербург выставки живописи"

- "выставочные залы Петербург"

- "art galleries Saint Petersburg"

- "все галереи Петербурга список"

- И т.д.

- Парсинг результатов поиска, извлечение названий и URL-ов
- Мерж LLM-списка и результатов поиска, дедупликация по названию (fuzzy)
- Результат: seed-список ~100-200 площадок

7.2. Шаг 2: Обогащение каталога площадок

Для каждой площадки из seed-списка:

- Загрузить главную страницу сайта
- LLM-экстракция: адрес, часы работы, описание, контакты, ссылка на раздел «Выставки»
- Геокодинг адреса (Яндекс Geocoder API или Nominatim) → `geo_lat`, `geo_lon`
- Скачать логотип / фото фасада → `image_url`
- Сохранить в таблицу `Venue` + создать `Source` с типом `website_museum` и URL раздела выставок

7.3. Шаг 3: Обнаружение телеграм-каналов

Процесс:

- Запрос к `tgstat.ru` (если доступен API) или ручной веб-поиск: каналы с ключевыми словами:
 - искусство Петербург, выставки СПб, галерея, музей Петербург
 - Также: каналы конкретных площадок из каталога
- Для каждого найденного канала:
 - Забрать описание + последние 30 постов
 - LLM оценивает релевантность: «Этот канал публикует информацию о выставках изобразительного искусства в Санкт-Петербурге?»
 - Оценка 0-1, порог 0.5 для включения
- Ранжирование по: релевантность × количество подписчиков × частота публикаций
- Топ-N каналов → таблица `Source` с типом `telegram_channel`

7.4. Шаг 4: Каталог агрегаторов

Фиксированный список (`kudago`, `afisha`, `sobaka`, `timeout`, `the-village`) — вносим вручную, т.к. их мало.

Для каждого:

- Определяем URL раздела «Выставки / СПб»

- Проверяем наличие API (kudago имеет публичный REST API)
- Для остальных – анализ структуры страницы (LLM помогает определить CSS-селекторы или паттерны)

- создаём Source с scraper_config

7.5. Еженедельное обновление

Каждое воскресенье:

- Повторить веб-поиск на предмет новых площадок/каналов
- Сравнить с текущим каталогом
- Новые кандидаты → в админку на подтверждение
- Неактивные источники (нет обновлений 30+ дней) → пометить для проверки

8. Телеграм-канал (News Feed)

8.1. Общие параметры

Параметр	Значение
Тип	Публичный канал
Бот	Dedicated bot, публикует от имени канала
Частота публикаций	Ежедневно, после завершения пайплайна (утро)
Формат поста	Изображение + текст + inline-кнопка

8.2. Типы публикаций

- **Новая выставка** – пост с обложкой, кратким описанием, кнопкой «Подробнее»
- **Обновление** – изменение дат, площадки (редкий тип)
- **Еженедельный дайджест** – «Заканчиваются на этой неделе», pinned-пост по воскресеньям

8.3. Inline-кнопки

Кнопка	Действие
Подробнее	Ссылка на страницу выставки на веб-портале

(На фазе 2 добавятся: «Хочу пойти», «Не интересно», «В календарь»)

9. Веб-портал

9.1. Архитектурный подход

SSG (Static Site Generation) + API для поиска.

- Основной контент (страницы выставок, площадок, художников, календарь) — генерируется статически после ежедневного пайплайна
- Поиск и фильтрация — лёгкий API-бэкенд
- Мобильная адаптивность (responsive design) — обязательна

9.2. Страницы и разделы

9.2.1. Главная (/)

- Блок «Новые выставки» — последние добавленные
- Блок «Заканчиваются скоро» — топ-5 событий, заканчивающихся в ближайшие 7 дней
- Блок «Сейчас проходят» — общее количество + ссылка на каталог

9.2.2. Каталог выставок (/exhibitions)

- Список выставок с фильтрами:
 - Статус: **идёт** / скоро / анонс
 - Теги (жанр, стиль)
 - Площадка
 - Художник
- Сортировка: по дате начала, по дате окончания, по дате добавления
- Карточки: изображение, название, площадка, даты, теги

9.2.3. Страница выставки (/exhibitions/{slug})

- Полное описание
- Изображение
- Площадка (ссылка)
- Даты
- Художники (ссылки)
- Теги
- Источники информации

9.2.4. Каталог площадок (/venues)

- Список площадок с фильтром по типу
- Карточки: название, тип, адрес, кол-во текущих выставок

9.2.5. Страница площадки (/venues/{slug})

- Описание, адрес, часы работы, фото
- Текущие выставки
- Прошедшие выставки (архив)

9.2.6. Каталог художников (/artists)

- Алфавитный список
- Поиск по имени

9.2.7. Страница художника (/artists/{slug})

- Биография, фото
- Связанные выставки (текущие + архив)

9.2.8. Календарь (/calendar)

- Calendar view (месяц) — визуализация выставок по датам
- Ось Y — площадки, ось X — даты (timeline view, gantt-подобный)
- Read-only, без взаимодействия

9.2.9. Заканчиваются скоро (/ending-soon)

- Список выставок, заканчивающихся в ближайшие 7 дней
- Сортировка по дате окончания (ближайшие первые)

9.2.10. Поиск (/search?q=...)

- Полнотекстовый поиск по: названиям выставок, площадкам, художникам, описаниям, тегам
- Результаты с группировкой по типу (выставки / площадки / художники)

9.3. SEO

- Канонические URL для всех сущностей
- Meta-теги (title, description, og:image)
- Structured data (JSON-LD): Event, Place, Person
- Sitemap.xml, robots.txt

10. Админ-интерфейс

10.1. Доступ

Простая авторизация (логин/пароль или Telegram OAuth). На фазе 1 – один-два администратора.

10.2. Разделы

10.2.1. Дашборд

- Всего выставок: running / announced / ended
- Собрано сегодня: raw events → relevant → published
- Источники: активных / с ошибками
- Очередь модерации: N событий ждут проверки
- График: события за последние 30 дней (собрано / опубликовано / отклонено)

10.2.2. Единая очередь задач (Pending Queue)

Единый интерфейс для всех элементов, требующих внимания администратора. Объединяет все типы pending-задач в одном месте с фильтрацией по типу:

Тип задачи	Описание	Действия
Модерация выставки	Новая выставка ожидает проверки	Approve / Reject / Edit & Approve
Новая площадка	Площадка создана автоматически, требует подтверждения	Confirm / Edit / Delete
Новый художник	Художник создан автоматически	Confirm / Edit / Merge с существующим
Новый источник	Кандидат обнаружен при еженедельном сканировании	Add / Ignore
Проблемный источник	Источник не отвечает 3+ дня	Retry / Disable / Delete
Возможный дубликат	Дедупликация не уверена (confidence 0.5-0.8)	Merge / Keep separate

Тип задачи	Описание	Действия
Конфликт данных	Противоречивые данные из разных источников	Выбрать верный вариант

Для каждой задачи отображается:

- Тип и приоритет
- Дата создания
- Контекст (сырой контент, нормализованный результат, предложенные связи)
- Кнопки действий

Счётчик нерешённых задач по типам отображается на дашборде и в виде badge на вкладке.

10.2.3. Очередь модерации выставок (подраздел Pending Queue)

- список RawEvent со статусом normalized + moderation_status = pending
- Для каждого: сырой контент, нормализованный результат, предложенные связи (площадка, художники)
- действия: Approve / Reject / Edit & Approve
- При Edit – возможность исправить любые поля перед публикацией

10.2.4. Управление выставками

- CRUD выставок
- Ручное создание выставки
- Изменение статуса, скрытие/показ
- Мерж дубликатов

10.2.5. Управление площадками

- CRUD площадок
- Список площадок pending (созданных автоматически, но не подтверждённых)
- Confirm / Edit / Delete

10.2.6. Управление художниками

- CRUD художников
- Мерж дубликатов (один художник мог быть создан под разными вариантами имени)

10.2.7. Управление источниками

- Список всех источников с метриками:
 - Статус (active / error / disabled)
 - Последний успешный сбор
 - Всего событий / релевантных (за всё время и за последний месяц)
 - Reliability score
- Добавление нового источника
- Отключение / удаление источника
- Ручной запуск сбора для конкретного источника

10.2.8. Логи пайплайна

- Лог последнего запуска: сколько собрано, отфильтровано, нормализовано, дедуплицировано
- Ошибки с деталями
- Использование LLM API: количество запросов, токенов, стоимость

11. Архитектура системы

11.1. Компоненты

Сервисы приложения:

Сервис	Функции
Scraper Service	HTTP fetch, Telethon client, Scheduler (cron)
Processor Service	LLM Relevance, Normalize, Tags, Enrichment, Dedup
Publisher Service	TG Bot, Feed generation, Pin/Unpin, SSG trigger
Web App (Frontend)	SSG pages, Search UI, Calendar, Admin UI

Инфраструктура:

Компонент	Назначение
PostgreSQL	venues, artists, exhibitions, sources, raw_events, feed_posts
Object Storage (S3 / MinIO)	Изображения (постеры выставок, фото площадок)
LLM API Gateway	Единый интерфейс к Claude / GPT, retry, fallback, мониторинг
Task Queue (Redis + Celery)	Оркестрация пайплайна, расписание, retry

11.2. Описание компонентов

Компонент	Ответственность
Scraper Service	Ежедневный сбор данных из всех источников. Планировщик (cron). HTTP-клиент, Telethon-клиент. Сохранение в RawEvent.
Processor Service	LLM-пайплайн: фильтрация → нормализация → тегирование → дедупликация → создание/обновление Exhibition. Модерация.
Publisher Service	Генерация постов. Публикация в Telegram-канал через Bot API. Триггер SSG-rebuild для веб-портала.
Web App	SSG-фронтенд (каталог, календарь, поиск). Админ-панель (SPA). API для поиска и админки.
PostgreSQL	Основное хранилище: HCI, raw events, feed posts.
Object Storage	Хранение изображений (постеры выставок, фото площадок). S3-совместимое.
LLM API Gateway	Абстракция над API провайдеров LLM. Retry, fallback, мониторинг расхода.
Task Queue	Оркестрация этапов пайплайна. Обеспечение порядка выполнения, retry, мониторинг.

11.3. Поток данных (ежедневный цикл)



03:00 Cron запускает Scraper Service

- └ Для каждого Source → скачать контент → RawEvent (status: new)
- └ ~03:30 Все источники обработаны

03:30 Cron запускает Processor Service

- └ RawEvent (new) → LLM Relevance → (filtered)
- └ RawEvent (relevant) → LLM Normalize → (normalized)
- └ Normalized → Dedup → Exhibition create/update
- └ Exhibition → Moderation queue (или auto-approve)

[Ручная модерация – в течение дня через админку]

После модерации / автоматически:

- └ Publisher: генерация FeedPost → публикация в Telegram
- └ Publisher: триггер SSG rebuild → обновление веб-портала

10:00 (Воскресенье) Publisher: дайджест «Заканчиваются» → Telegram + pin

00:00 Cron: обновление статусов Exhibition (announced→running→ending_soon→ended)



11.4. LLM API Gateway

Поддерживаемые провайдеры:

- Anthropic Claude (основной)
- OpenAI GPT-4o (fallback)

Стратегия:

- Primary: Claude (лучше для длинных текстов и экстракции на русском)
- Fallback: GPT-4o (при недоступности основного)
- Для простых задач (классификация relevance): можно использовать более лёгкие модели (Claude Haiku / GPT-4o-mini) для экономии

Мониторинг:

- Логирование каждого запроса: модель, токены input/output, latency, стоимость
- Дневной/месячный отчёт по расходам в админке

12. Стек технологий

12.1. Рекомендованный стек

Слой	Технология	Обоснование
Язык бэкенда	Python 3.12+	Лучшая экосистема для скрейпинга, LLM API, Telethon
Веб-фреймворк	FastAPI	Async, быстрый, OpenAPI spec из коробки
ORM	SQLAlchemy 2.0 + Alembic	Стандарт для Python, миграции
БД	PostgreSQL 16	jsonb, full-text search, надёжность
Полнотекстовый поиск	PostgreSQL FTS (pg_trgm)	Достаточно для масштаба 100-200 площадок. При росте → Meilisearch
Task Queue	Celery + Redis	Стандарт для Python, cron-расписание через Celery Beat
Скрейпинг	httpx + BeautifulSoup	Async HTTP + парсинг HTML
Telegram (чтение каналов)	Telethon	User-клиент для чтения публичных каналов
Telegram (публикация)	aiogram 3	Async bot framework для публикации в канал
LLM API	LiteLLM	Единый интерфейс к Claude, GPT, etc.
Object Storage	MinIO (self-hosted) или S3	S3-совместимый, хранение изображений
Frontend (портал)	Next.js 14 (App Router, SSG)	SSG + ISR, React, отличная мобильная разработка
Frontend (админка)	React Admin или встроена в Next.js	Быстрая разработка CRUD-интерфейсов
CSS	Tailwind CSS	Быстрая адаптивная верстка
Деплой	Docker Compose	Простой single-server деплой на фазу 1
CI/CD	GitHub Actions	Автотесты, деплой
Мониторинг	Sentry + простой дашборд	Ошибки + метрики пайплайна

12.2. Структура репозитория (предварительная)

```
`
culture-spb/
├─ backend/
│ └─ app/
│   │ └─ models/ # SQLAlchemy models
│   │ └─ scrapers/ # Scraper implementations per source type
│   │ └─ processor/ # LLM pipeline (relevance, normalize, dedup)
│   │ └─ publisher/ # Telegram publisher, SSG trigger
│   │ └─ api/ # FastAPI routes (search, admin)
│   │ └─ tasks/ # Celery tasks
│   └─ config.py
├─ migrations/ # Alembic
├─ tests/
└─ pyproject.toml
├─ frontend/
│ └─ app/ # Next.js App Router
│   │ └─ page.tsx # Home
│   │ └─ exhibitions/
│   │ └─ venues/
│   │ └─ artists/
│   │ └─ calendar/
│   │ └─ ending-soon/
│   │ └─ search/
│   └─ admin/
├─ components/
└─ package.json
├─ scripts/
│ └─ bootstrap_venues.py # Seed каталога площадок
│ └─ discover_channels.py # Обнаружение TG-каналов
│ └─ discover_sources.py # Обнаружение агрегаторов
├─ docker-compose.yml
├─ .env.example
└─ README.md
`
```

13. Нефункциональные требования

13.1. Производительность

Метрика	Целевое значение
Полный ежедневный пайплайн	≤ 2 часа
Загрузка веб-страницы (SSG)	< 1 сек (TTFB)
Поиск	< 500 мс
SSG rebuild	< 5 мин

13.2. Надёжность

- Пайплайн должен быть идемпотентным (повторный запуск не создаёт дубликатов)
- При падении одного источника – остальные продолжают работать
- Алертинг при: 3+ дней без успешного сбора, аномальное количество нерелевантных событий

13.3. Масштабируемость

- Фаза 1: single server (Docker Compose)
- Архитектура позволяет горизонтальное масштабирование Scraper и Processor (через task queue)
- Добавление нового города = добавление Sources + Venues, без изменения кода

13.4. Безопасность

- Админка за авторизацией
- API keys LLM-провайдеров – в секретах (.env), не в коде
- Telegram user session – хранить безопасно (encrypted)
- Rate limiting на публичный API (поиск)

13.5. Стоимость LLM API – детальная оценка

Допущения:

- ~200 источников, ~300 raw events/день (многие нерелевантные)
- ~60% отсеивается на этапе фильтрации → ~120 релевантных событий/день
- ~50% дубликаты → ~60 уникальных выставок/день (новых + обновлений)
- Средний размер raw content: ~2,000 токенов (input)
- Средний размер ответа LLM: ~500 токенов (output)
- 30 дней в месяце

LLM-вызовы на одно сырое событие (worst case – все этапы):

Этап	Input tokens	Output tokens	Кол-во вызовов/ день
Фильтрация (все 300 events)	~2,000	~100	300
Нормализация (120 relevant)	~2,000	~500	120
Обогащение (60 из вторичных источников)	~3,000	~500	60
Дедупликация-LLM (30 неочевидных пар)	~1,500	~200	30
Итого в день	~510		
	вызовов		

Суммарные токены в день:

- Input: $300 \times 2,000 + 120 \times 2,000 + 60 \times 3,000 + 30 \times 1,500 = 600K + 240K + 180K + 45K = \sim 1,065K$ input tokens/день
- Output: $300 \times 100 + 120 \times 500 + 60 \times 500 + 30 \times 200 = 30K + 60K + 30K + 6K = \sim 126K$ output tokens/день

Суммарные токены в месяц:

- Input: $\sim 32M$ tokens/мес
- Output: $\sim 3.8M$ tokens/мес

Стоимость по моделям (цены на март 2026, приблизительные):

Модель	Input \$/1M	Output \$/1M	Input/мес	Output/мес	Итого/мес
Claude 3.5 Haiku	\$0.80	\$4.00	\$25.6	\$15.2	\$40.8
Claude 3.5 Sonnet	\$3.00	\$15.00	\$96.0	\$57.0	\$153.0
Claude Opus 4	\$15.00	\$75.00	\$480.0	\$285.0	\$765.0
GPT-4o mini	\$0.15	\$0.60	\$4.8	\$2.3	\$7.1
GPT-4o	\$2.50	\$10.00	\$80.0	\$38.0	\$118.0
GPT-4.1 mini	\$0.40	\$1.60	\$12.8	\$6.1	\$18.9
GPT-4.1	\$2.00	\$8.00	\$64.0	\$30.4	\$94.4

Рекомендованная стратегия (оптимум цена/качество):

Этап	Модель	Обоснование
Фильтрация (самый массовый)	GPT-4o mini / Claude Haiku	Простая классификация, дешёвая модель достаточно
Нормализация	Claude 3.5 Sonnet / GPT-4o	Нужно хорошее понимание русского текста
Обогащение	Claude 3.5 Sonnet / GPT-4o	Экстракция из HTML, нужна качественная модель
Дедупликация	GPT-4o mini / Claude Haiku	Сравнение двух текстов, дешёвая модель ok

Оценка при оптимальной стратегии:

- Фильтрация (GPT-4o mini): $300 \times 30 = 9,000$ вызовов/мес $\rightarrow \sim \$2$
- Нормализация (Claude Sonnet): $120 \times 30 = 3,600$ вызовов/мес $\rightarrow \sim \$15$
- Обогащение (Claude Sonnet): $60 \times 30 = 1,800$ вызовов/мес $\rightarrow \sim \$12$
- Дедупликация (GPT-4o mini): $30 \times 30 = 900$ вызовов/мес $\rightarrow \sim \$0.5$
- LLM итог: $\sim \$30$ /мес при оптимальной стратегии

13.6. Общая стоимость эксплуатации

Статья	Оценка/мес
LLM API (оптимальная стратегия)	\$25–40
LLM API (всё на Claude Sonnet)	\$120–160
Сервер (VPS, 4 CPU / 8GB RAM)	\$20–50
Object Storage (изображения)	\$5–10

Статья	Оценка/мес
Домен	~\$1 (~\$12/год)
Итого (оптимально)	\$50–100/мес
Итого (premium модели)	\$150–220/мес

14. Фаза 2+ — Дорожная карта

Фаза 2: Пользовательские функции

Фокус: Персонализация и взаимодействие.

- **Авторизация** — Telegram Login / email
- **Личный календарь** — добавление выставок в «Хочу пойти»
- **Шеринг календаря** — публичная ссылка на свой план
- **Напоминания** — персональные напоминания о выбранных выставках (Telegram-бот в ЛС)
- **Отзывы** — короткий отзыв + оценка после посещения
- **Отметка «Не интересно»** — исключение из рекомендаций
- **Inline-кнопки в Telegram** — «Хочу пойти» / «Не интересно» прямо в посте канала
- **Карта** — интерактивная карта площадок с текущими выставками

Фаза 3: Рекомендации и аналитика

- **Рекомендации** — на основе тегов, лайков, посещений: «Вам может понравиться»
- **Аналитика популярности** — какие выставки чаще добавляют, какие площадки популярнее
- **AI-ассистент** — чат-бот: «Что посмотреть в эти выходные?», «Есть ли выставки импрессионистов?»

Фаза 4: Масштабирование

- **Мультигородность** — Москва, Казань, другие города
- **Расширение тематики** — фотовыставки, перформансы, арт-ярмарки
- **Мультиязычность** — английская версия портала
- **Партнёрства** — интеграция с билетными системами, коллаборации с площадками

Приложения

А. Примеры LLM-промптов

Полные тексты промптов для каждого этапа пайплайна находятся в разделе 6 (этапы 2, 3, 4).

В. Первичный список площадок

Будет сформирован автоматически (раздел 7) и сохранён в `culture-spb/data/venues_seed.json`.

С. Первичный список телеграм-каналов

Будет сформирован автоматически (раздел 7) и сохранён в `culture-spb/data/channels_seed.json``.

Документ будет обновляться по мере уточнения требований. Версионирование через Git.